Spring Boot — Email Service Notes

Normal Approach • HTML Template Approach

1. Theory — How Email Works in Spring Boot

1.1 Core Dependencies

Spring Boot email support is provided by the spring-boot-starter-mail dependency, which internally uses Jakarta Mail (formerly JavaMail). You configure it via application.properties.

```
<!-- pom.xml -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-mail</artifactId>
</dependency>
```

```
# application.properties
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=your@gmail.com
spring.mail.password=your_app_password
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
```

1.2 Key Interfaces & Classes

Class / Interface	Package	Purpose
JavaMailSender	org.springframework.mail.javamail	Main interface to send emails; autowired by Spring
SimpleMailMessage	org.springframework.mail	Simple plain-text email (to, from, subject, body)
MimeMessage	jakarta.mail.internet	Supports HTML, attachments, inline images
MimeMessageHelper	org.springframework.mail.javamail	Helper to build MimeMessage easily (optional)

1.3 SimpleMailMessage vs MimeMessage

Feature	SimpleMailMessage	MimeMessage
Content Type	Plain text only	Plain text, HTML, mixed
Attachments	✗ Not supported	✓ Supported
HTML body	✗ Not supported	✓ Supported
Complexity	Simple / low boilerplate	More control required
Use case	Notifications, alerts	Branded emails, rich content

2. Normal Approach — SimpleMailMessage

2.1 What It Does

Sends a plain-text email using SimpleMailMessage. This is the simplest approach—no HTML, no attachments, just to/from/subject/body. Best for internal alerts or simple notifications.

2.2 MailFormat DTO

MailFormat is a plain POJO (or record) that carries the email fields into the service.

```
// MailFormat.java
@Data
public class MailFormat {
    private String to;
    private String subject;
    private String body;
}
```

2.3 EmailService Code

```
@Slf4j
@Service
public class EmailService {

    @Autowired
    private JavaMailSender javaMailSender;

    public void sendEmail(MailFormat mailFormat) {
        try {
            SimpleMailMessage simpleMailMessage = new SimpleMailMessage();
            simpleMailMessage.setFrom("khatiwadasandesh501@gmail.com");
            simpleMailMessage.setTo(mailFormat.getTo());
            simpleMailMessage.setSubject(mailFormat.getSubject());
            simpleMailMessage.setText(mailFormat.getBody());
            javaMailSender.send(simpleMailMessage);
        } catch (Exception e) {
            log.error("Couldn't send message to {}", mailFormat.getTo(), e);
        }
    }
}
```

2.4 Line-by-Line Explanation

- **@Slf4j** — Lombok annotation; injects a log field (uses SLF4J logger). Used for log.error(...).
- **@Service** — Marks this class as a Spring service component; auto-detected by component scan.
- **@Autowired JavaMailSender** — Spring injects the mail sender bean configured via application.properties.
- **SimpleMailMessage** — Lightweight wrapper for plain-text email fields.
- **javaMailSender.send()** — Hands off the message to the SMTP server.

- **try-catch + log.error** — Catches SMTP/network failures and logs them without crashing the application.

2.5 Pros & Cons

✅Pros	❌Cons
Very simple, minimal code	Plain text only — no styling
No template files needed	Cannot embed images or attachments
Good for internal alerts	Not suitable for user-facing branded emails

3. HTML Template Approach — MimeMessage

3.1 What It Does

Sends an HTML email by loading a .html file from the classpath (src/main/resources/templates/), replacing placeholders like ${name} with real values, and setting the content type to text/html. This enables styled, branded emails.

3.2 HTML Template File

Place your HTML file at: src/main/resources/templates/template.html

```
<!-- templates/template.html -->
<!DOCTYPE html>
<html>
  <body style="font-family: Arial, sans-serif;">
    <h2>Hello, ${name}!</h2>
    <p>Your attendance has been recorded successfully.</p>
    <p>— Attendance Management System</p>
  </body>
</html>
```

3.3 MailServiceImpl Code

```
@Slf4j
@Service
public class MailServiceImpl implements MailService {

    private final JavaMailSender javaMailSender;

    public MailServiceImpl(JavaMailSender javaMailSender) {
        this.javaMailSender = javaMailSender;
    }

    @Override
    public void mailSender(String to, String username)
            throws MessagingException, IOException {
```

```
        MimeMessage message = javaMailSender.createMimeMessage();
        message.setFrom(new InternetAddress("Khatiwadasandesh501@gmail.com"));
        message.setRecipients(MimeMessage.RecipientType.TO, to);
        message.setSubject("Attendance Management System");

        ClassPathResource htmlPage = new
ClassPathResource("templates/template.html");
        String htmlTemplate = new String(
            htmlPage.getInputStream().readAllBytes(),
            StandardCharsets.UTF_8
        );
        htmlTemplate = htmlTemplate.replace("${name}", username);
        message.setContent(htmlTemplate, "text/html; charset=utf-8");

        javaMailSender.send(message);
        log.info("Message sent to : {}", to);
    }
}
```

3.4 Line-by-Line Explanation

- **Constructor injection** — Uses constructor-based DI instead of @Autowired field injection. This is preferred for testability and immutability (final field).
- **implements MailService** — Enforces a contract via interface; allows swapping implementations without changing callers.
- **createMimeMessage()** — Creates a MIME-capable message object that supports HTML and attachments.
- **InternetAddress** — Parses and validates an email address string in RFC 822 format.
- **setRecipients(...RecipientType.TO...)** — Sets the TO recipients. Can also use CC and BCC RecipientType values.
- **ClassPathResource** — Spring utility to load files from the classpath (resources folder) without hardcoding absolute paths.
- **readAllBytes() + StandardCharsets.UTF_8** — Reads the entire HTML file into a String with explicit UTF-8 encoding.
- **htmlTemplate.replace("${name}", username)** — Manual placeholder substitution. For complex templates, Thymeleaf or Freemarker is recommended.
- **setContent(..., "text/html; charset=utf-8")** — Sets the email body as HTML. Without this, the raw HTML tags would appear as plain text.
- **log.info(...)** — Logs successful sends (unlike the normal approach which only logs errors).

3.5 Pros & Cons

✅Pros	❌Cons
Full HTML/CSS styling	Requires managing template files
Dynamic placeholders via replace()	Manual replace() is fragile for many variables
Professional user-facing emails	Throws checked exceptions (MessagingException, IOException)

Constructor injection = more testable	More boilerplate than SimpleMailMessage

4. Side-by-Side Comparison

Aspect	Normal (SimpleMailMessage)	HTML Template (MimeMessage)
Message Type	SimpleMailMessage	MimeMessage
Content	Plain text only	HTML + CSS
Injection	@Autowired (field)	Constructor injection
Template	None — text set directly	External .html file in classpath
Dynamic values	Via MailFormat DTO fields	String.replace() on template
Exception handling	try-catch (unchecked)	Throws checked exceptions
Logging	log.error on failure	log.info on success
Best for	Internal alerts, simple notifications	User-facing styled emails

5. Quick Tips & Common Pitfalls

- **Gmail App Password:** If using Gmail SMTP, use an App Password (not your Google account password). Enable 2FA first, then generate one at myaccount.google.com/apppasswords.
- **Port 587 vs 465:** Port 587 uses STARTTLS (upgrades plain connection to TLS). Port 465 uses SSL/TLS from the start. Most providers prefer 587.
- **HTML charset:** Always pass "text/html; charset=utf-8" to setContent() to avoid garbled special characters.
- **Thymeleaf for complex templates:** For emails with loops, conditionals, or many variables, use spring-boot-starter-thymeleaf and TemplateEngine instead of manual replace().
- **Never hardcode credentials:** Keep email/password in application.properties or environment variables, not in source code.
- **Test with MailHog:** During development, use MailHog (a fake SMTP server) to catch emails without actually sending them.

End of Notes